Vibe vs. Verified

Token-efficient AI development with the Cursor and Claude Code CLIs

Cheat sheet

The goal isn't fewer tokens. It's more correct, verified progress per minute, per token and per prompt. Plan to keep the agent on target; manage context to keep it cheap.

1 The loop

Aim
Six-part prompt. Say what you already know.

Plan
Make its thinking visible. Review it; then it's the contract.

Build
Only what the plan says. No drive-by refactors.

Prove
Narrowest test → typecheck → full suite once.

Reset
New goal → /clear. Same goal, long chat → compact.

2 The six-part prompt

Part	Answers
Goal	What observable result exists afterwards?
Acceptance	How do we know it's right? Include the edge cases.
Context	Which files, symbols, tests, docs, logs?
Constraints	What must not change? What not to read?
Plan?	Plan first (and stop), or straight to the edit?
Verify	The exact command that proves it.

```
Goal: cap WHOLESALE discounts at £500. Retail
unchanged.
Acceptance: retail £600 → £600;
  wholesale £400 → £400, £600 → £500
Context: @apply-discount.ts @pricing-policy.ts
  @wholesale-discount.test.ts
Constraints: no promotions/ edits, no floats,
  cap lives in PricingPolicy, don't weaken tests
Plan: ≤ 8 lines, then stop for my review
Verify: npm test -- tests/pricing/wholesale-…
```

3 Planning keeps the agent on target

Every agent plans. Plan mode makes it plan where you can see it, so a wrong assumption costs one line to fix instead of a correction spiral, rework or a new session. Skip it for one obvious line.

```
Ask the plan for
Goal          one line, in your words
Change        the files it will edit
Don't touch   folders and files out of scope
Approach      where the logic goes, and why
Tests         cases, including the edge case
Verify        the exact commands
Open Qs       anything it had to guess
```

Then treat it as the contract

- ✓ Approve only what you've actually read
- ✓ Build only what's in the plan
- ✓ New evidence → stop and re-plan, don't patch around it
- ✓ "Don't touch" is what stops drive-by refactors
- ✓ Open questions are where bugs hide: answer them

1 · Plan
Plan mode (Shift+Tab in both CLIs). It explores; the window fills.

2 · Save
"Write the approved plan to `plans/<task>.md`."

3 · Clear
`/clear`. Exploration noise gone; the plan file and repo remain.

4 · Build
"Implement `plans/<task>.md`." A fresh, small window.

The plan file doubles as your handoff. A plan that just restates your prompt cost tokens and bought nothing.

Vibe habits that burn tokens

- ✗ "Fix it" prompts with no file, rule or test
- ✗ One chat for every goal of the day
- ✗ Attaching a whole folder "just in case"
- ✗ Running the full test suite every turn
- ✗ Five "no, not that" corrections in a row
- ✗ The strongest model for a rename

Keep the window lean

Context is a working set, not a warehouse: all of it is re-read every turn

4 Same habits, two CLIs

Habit	Claude Code CLI · claude	Cursor CLI · agent
Plan before editing	**Shift+Tab** until the footer says plan mode	**Shift+Tab** cycles Agent → Plan → Ask · `/plan` · `/ask`
Point at the code	`@path/to/file`, then name the symbol	`@path/to/file` or a folder, then name the symbol
See your context	`/context` · `/statusline` showing context %	Custom status line with context % (`statusLine` in `~/.cursor/cli-config.json`)
See your spend	`/usage` or `/cost`	Usage on the cursor.com dashboard
Same goal, long chat	`/compact <what to keep>`	`/summarize` (alias `/compress`)
New goal	`/clear`	`/clear` (alias `/new-chat`)
Undo a wrong turn	**Esc** to interrupt · **Esc Esc** or `/rewind`	`/rewind` · **Ctrl+R** to review changes
Right model for the job	`/model`	`/model`
Explore without bloat	Ask it to use a subagent to investigate	`/fork` to explore, bring the answer back
Trim always-on tools	`/mcp`	`/mcp list`
Scripted, one-shot	`claude -p "…"`	`agent -p "…"`

5 Continue, compact or clear?

Situation	Do	Keeps
Same goal, small	Continue	Everything
Same goal, long chat	`/compact` · `/summarize`	A summary you steer
New goal	`/clear`	Repo + plan file
Wrong turn	`/rewind`	All before it

Reset when… the goal changes · it forgets a constraint you gave · it retries a failed fix · old logs no longer matter · context passes ~50% mid-task.
Compact well: at a boundary, not mid-edit · say what to keep · restate the key constraint afterwards · don't wait for auto-compact to choose for you.

Clearing isn't losing work: **code, tests, docs, git and your plan file are the memory**; the transcript is scaffolding. No plan file? Write a ≤ 200-word handoff first.

6 Progressive disclosure for agent docs

0 **Always loaded** · CLAUDE.md / AGENTS.md: commands, non-negotiables, pointers — **every turn · keep to ~1 screen**

1 **Routing map** · docs/architecture.md: which file owns what — **when locating code**

2 **Scoped rules** · .cursor/rules/*.mdc with globs · nested CLAUDE.md — **when matching files are touched**

3 **Deep docs** · decision records, runbooks, skills, specs — **only when changing that rule**

Write **pointers, not paragraphs**: *"Read docs/architecture.md only when you need to locate code."* After a bad session, put the lesson at the **narrowest layer** where it will still be found, not in CLAUDE.md. Disable MCP servers you aren't using: their tool definitions load every session.

7 Pocket checklist

- **BEFORE** — One goal · acceptance criteria · name the files · plan if anything is uncertain · note context %
- **DURING** — Build only what the plan says · narrowest test first · re-plan when evidence changes
- **SWITCHING** — Save the plan or a handoff to a file · same goal → compact · new goal → /clear
- **AFTER** — Verify · nothing unrelated changed · lessons go to the narrowest doc layer